

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Національний університет «Запорізька політехніка»

Факультет комп'ютерних наук і технологій
(повне найменування факультету)

Кафедра програмних засобів
(повне найменування кафедри)

Пояснювальна записка

до дипломного проекту (роботи)

бакалавр

(ступінь вищої освіти)

на тему Розроблення програмного забезпечення інтернет-магазину
вантажівок

Development of Software for an Online Truck Store

Виконав(ла): студент(ка) 4 курсу, групи КНТ-122
Спеціальності 121 Інженерія програмного
(код і найменування спеціальності)
забезпечення

Освітня програма (спеціалізація)
Інженерія програмного забезпечення

ОНИЩЕНКО О.А.

(ПРИЗВИЩЕ та ініціали)

Керівник СТЕПАНЕНКО О.О.

(ПРИЗВИЩЕ та ініціали)

Рецензент ДЬЯЧУК Т.С.

(ПРИЗВИЩЕ та ініціали)

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Національний університет «Запорізька політехніка»
(повне найменування закладу вищої освіти)

Факультет КНТ
Кафедра програмних засобів
Ступінь вищої освіти бакалавр
Спеціальність 121 Інженерія програмного забезпечення
(код і найменування)
Освітня програма (спеціалізація) Інженерія програмного забезпечення
(назва освітньої програми (спеціалізації))

ЗАТВЕРДЖУЮ

Завідувач кафедри ПЗ, д.т.н, проф.
Сергій СУББОТІН
“ ” 2026 року

З А В Д А Н Н Я

НА ДИПЛОМНИЙ ПРОЄКТ (РОБОТУ) СТУДЕНТА(КИ)

ОНИЩЕНКО Олега Антоновича

(ПРІЗВИЩЕ, ім'я, по батькові)

1. Тема проєкту (роботи) Розроблення програмного забезпечення інтернет-магазину вантажівок. Development of Software for an Online Truck Store
керівник проєкту (роботи) к.т.н., доцент, СТЕПАНЕНКО Олександр Олексійович,

(науковий ступінь, вчене звання, ПРІЗВИЩЕ, ім'я, по батькові)

затверджені наказом закладу вищої освіти від “ 14 ” січня 2026 року № 1

2. Строк подання студентом проєкту (роботи) 10 червня 2026 року

3. Вихідні дані до проєкту (роботи) рекомендована література, технічне завдання

4. Зміст розрахунково-пояснювальної записки (перелік питань, які потрібно розробити) 1. Аналіз проблеми та постановка завдань дослідження. 2. Матеріали і методи. 3. Опис програми. 4. Експлуатація, тестування та експериментальне дослідження програми.

5. Перелік графічного матеріалу (з точним зазначенням обов'язкових креслень, кількість слайдів, плакатів)

Слайди презентації

6. Консультанти розділів проєкту (роботи)

Розділ	ПРИЗВИЩЕ, ініціали та посада консультанта	Підпис, дата	
		завдання видав	прийняв виконане завдання
1-4 Основна частина	СТЕПАНЕНКО О.О., доцент		
Нормоконтроль	БЄЛОВА А.В., ст. викладач		

7. Дата видачі завдання « 14 » січня 2024 року.

КАЛЕНДАРНИЙ ПЛАН

№ з/п	Назва етапів дипломного проєкту (роботи)	Строк виконання етапів проєкту (роботи)	Примітка
1	Постановка завдання роботи.	1 тиждень	Завдання, ТЗ
2	Аналіз предметної області.	1 тиждень	Розділ 1
3	Вибір мови програмування та інших технологій розробки.	2 тиждень	Розділ 2
4	Розробка архітектури програми.	2 тиждень	Розділ 3
5	Розробка програми.	3-4 тижні	Розділ 3, 4
6	Тестування та експериментальне дослідження програмного забезпечення.	5 тиждень	Розділ 4
7	Оформлення пояснювальної записки та документів до неї.	6 тиждень	Додатки
8	Нормоконтроль та рецензування.	7 тиждень	
9	Захист роботи.	8 тиждень	

Студент(ка)

_____ Олег ОНИЩЕНКО
(підпис) (Ім'я ПРИЗВИЩЕ)

Керівник проєкту (роботи)

_____ Олександр СТЕПАНЕНКО
(підпис) (Ім'я ПРИЗВИЩЕ)

РЕФЕРАТ

Пояснювальна записка до дипломної кваліфікаційної роботи бакалавра містить: 55 сторінок, 8 таблиць, 20 рисунків, 3 додатки, 6 джерел.

ІНТЕРНЕТ МАГАЗИН ВАНТАЖІВОК, МАГАЗИН ТОВАРІВ, ВЕБ-ЗАСТОСУНОК, NICEGUI, PYTHON.

Об'єкт проєктування – процес розробки програмного забезпечення про продаж вантажівок.

Предмет проєктування – методи та програмні засоби купівлі та продажу вантажівок.

Мета роботи – підвищення швидкості пошуку вантажних автомобілів шляхом розробки програмного забезпечення для автосалону вантажівок.

Матеріали, методи та технічні засоби – використання відкритих джерел інформації та відомих засобів розробки.

Результатом роботи має бути готова програма, пояснювальна записка, презентація роботи.

Програмний застосунок містить наступні функції:

- купівля вантажівки;
- продаж вантажівки;
- здійснення рейсів.

Застосунок може бути використаний на таких платформах:

- Windows;
- MacOS;
- Android;
- iOS.

ABSTRACT

The explanatory note to the bachelor's thesis contains: 55 pages, 8 tables, 20 figures, 3 appendices, 6 sources.

INTERNET TRUCK STORE, GOODS STORE, WEB APPLICATION, NICEGUI, PYTHON.

The object of the design is the process of developing software for the sale of trucks.

The subject of the design is the methods and software tools for buying and selling trucks.

The goal of the work is to speed up the search for trucks.

Materials, methods, and technical means: use of open sources of information and well-known development tools.

The result of the work should be a finished program, explanatory note, and project presentation.

The software application includes the following functions:

- truck purchase;
- truck sale;
- trip execution.

The application can be used on the following platforms:

- Windows;
- MacOS;
- Android;
- iOS.

ЗМІСТ

Пояснювальна записка.....	1
З А В Д А Н Н Я.....	2
РЕФЕРАТ.....	4
ABSTRACT.....	5
ЗМІСТ.....	6
ПЕРЕЛІК СКОРОЧЕНЬ ТА УМОВНИХ ПОЗНАК.....	9
ВСТУП.....	10
1 АНАЛІЗ ПРЕДМЕТНОЇ ОБЛАСТІ ТА ОГЛЯД АНАЛОГІВ.....	11
1.1 Вступ.....	11
1.2 Огляд існуючих програмних засобів.....	11
1.3 Порівняння застосунків.....	18
1.4 Вимоги до розробки.....	18
1.5 Постановка задачі.....	18
1.5 Висновки за розділом.....	18
2 РОЗРОБКА АРХІТЕКТУРИ ПРОГРАМИ.....	20
2.1 Вступ.....	20
2.2 Вибір інструментів розробки.....	20
2.3 Вибір і проектування бази даних.....	22
2.4 Висновки за розділом.....	24
3 ПРОГРАМНА РЕАЛІЗАЦІЯ.....	25
3.1 Вступ.....	25

3.2 Реалізація клієнтської частини.....	25
3.3 Реалізація серверної частини.....	28
3.4 Взаємодія клієнта та сервера.....	30
3.5 Висновки за розділом.....	31
4 ТЕСТУВАННЯ ПРОГРАМИ.....	32
4.1 Вступ.....	32
4.2 Призначення програми.....	32
4.3 Процес тестування.....	32
4.4 Висновок за розділом.....	34
5 КЕРІВНИЦТВО КОРИСТУВАЧА.....	36
5.1 Необхідні умови для виконання.....	36
5.2 Звертання до програми.....	36
ВИСНОВКИ.....	42
ПЕРЕЛІК ДЖЕРЕЛ ПОСИЛАННЯ.....	43
ДОДАТОК А.....	45
Вступ.....	46
Підстави для розробки.....	46
Призначення розробки.....	46
Вимоги до програми.....	46
Вимоги до програмної документації.....	47
ДОДАТОК Б.....	48
Текст головного файлу main.py.....	49

Текст додаткового модулю data.py.....	55
ДОДАТОК В.....	58

ПЕРЕЛІК СКОРОЧЕНЬ ТА УМОВНИХ ПОЗНАК

NiceGUI Бібліотека створення графічних інтерфейсів мовою Python

Python Високорівнева мова програмування з динамічною типізацією

SQLite Файлова реляційна база даних

ВСТУП

Перевезення є важливою галуззю в світі. Люди щодня мають потреби. Їм потрібно їсти, пити та вдягатися. Для цього існують магазини, де люди купують товари. Місцеві магазини потребують доставки товару. Ланцюг доставки приблизно виглядає так:

1. товар виробляється на заводах;
2. з заводів товар перевозиться на склади;
3. зі складів товари розвозяться по магазинах.

Аби працювати на ринку перевезень потрібно або працювати на вантажівці компанії, або придбати власну. Власну вантажівку можна придбати на різних вебсайтах. Кілька відомих брендів вантажівок:

- DAF;
- Volvo;
- Scania.

Кожна з цих компаній має веб-сайт, на якому можна придбати (або подивитися де можна придбати) власну вантажівку.

Програма корисна тим, що об'єднує в собі всі місця пошуку автомобілів і дозволяє користувачеві їх придбати.

1 АНАЛІЗ ПРЕДМЕТНОЇ ОБЛАСТІ ТА ОГЛЯД АНАЛОГІВ

1.1 Вступ

Вантажні перевезення є важливою галуззю у світі. Для вантажних перевезень потрібні вантажні автомобілі. Вантажні автомобілі бувають наступних типів [1]:

- тент;
- трал;
- бус;
- евакуатор;
- маніпулятор;
- рефрижератор.

Зі зростанням цифровізації в Україні ринок електронної комерції також зростає. Станом на 2023 рік обсяг ринку електронної комерції становить 6 мільярдів доларів. Основні чинники, які сприяють розвитку ринку електронної комерції є зростання довіри споживачів до онлайн покупок, розширення цифрової інфраструктури та вдосконалення логістичних систем. [2]

Знайдені застосунки є застосунками, які допомагають купити власний вантажний автомобіль. Вони є гарними прикладами гнучкого функціоналу та зручного графічного інтерфейсу користувача.

1.2 Огляд існуючих програмних засобів

1.2.1 Веб-застосунок Auto.RIA

Застосунок Auto.RIA[3] є популярним веб-застосунком для продажу та купівлі автомобілів. Він містить різні фільтри. Деякі з них нижче:

- наявність відео машини;
- регіон;
- чи була у ДТП;

- технічний стан;
- ціна.

Переваги:

- зрозумілий інтерфейс;
- швидка взаємодія;
- багато пропозицій.

Вигляд інтерфейсу веб-застосунку на рисунку 1.1 нижче:

Донат до донату – автівка на фронт! Подвоїмо кожну гривню!

Пошук авто в Україні

Всі

Вживані

Нові

Під пригон

Тип транспорту
Вантажівки

Тип кузова

Марка

Рік випуску

Перевірений VIN

Держ. номер

Вперше на AUTO.RIA

Відео

Електрокар

Продає AUTO.RIA

Trade-in

Регіон

Область

Ціна

Від

До

\$

€

грн.

Ціна з ПДВ

Можливий торг

Обмін на автомобіль

Стан

Участь в ДТП

Вантажівки X

Відео X

Авто в Україні X

Розмитнені X

Очистити все

Розширений пошук

268 пропозицій

Звичайне ↑↓



Mercedes-Benz Sprinter 2019

W907/W910

26 500 \$ · 1 152 750 грн

364 тис. км

Автомат

Дизель, 2.2 л

Рівне (Рівненська)

Доставка у ваше місто

Доступний кредит

Торг

#Автомобіль розмитнений і зроблений сертифікат,...

5 днів тому



Mercedes-Benz Sprinter 2020

W907/W910

75 000 \$ · 3 262 500 грн

94 тис. км

Автомат

Дизель, 3 л

Рівне (Рівненська)

Доставка у ваше місто

Доступний кредит

Торг

Автомобіль розмитнений зроблений сертифікат, готови...

5 днів тому

Пропозиція дня



Opel Combo Cargo 2021

I покоління

13 300 \$ · 578 550 грн

126 тис. км

Ручна / Механіка

Дизель, 1.5 л

Ковель

ДОСТАВКА АВТО ПО УКРАЇНІ МОЖЛИВА РОЗСТРОЧКА
Пригнаний з Німеччини. Двигун, ходова, кпп – працюють
чудово. Автомобіль у відмінному стані! Гарна гума...

Перевірений VIN

BO 7622 ET



Mercedes-Benz Sprinter 2001

W901/W905

7 450 \$ · 324 075 грн

361 тис. км

Не вказано

Підписатись на пошук

Рисунок 1.1 — Вигляд веб-застосунку

1.2.2 Веб-застосунок Volvo

Веб-сайт Volvo[4] є офіційним сайтом компанії. Корисною функцією є пошук автосалонів. Веб-застосунок має світлий інтерфейс та можливість зібрати власну вантажівку у конфігураторі.

Переваги:

- зрозумілий інтерфейс;
- наявність пошуку автосалонів.

Вигляд застосунку на рисунку 1.2 нижче:

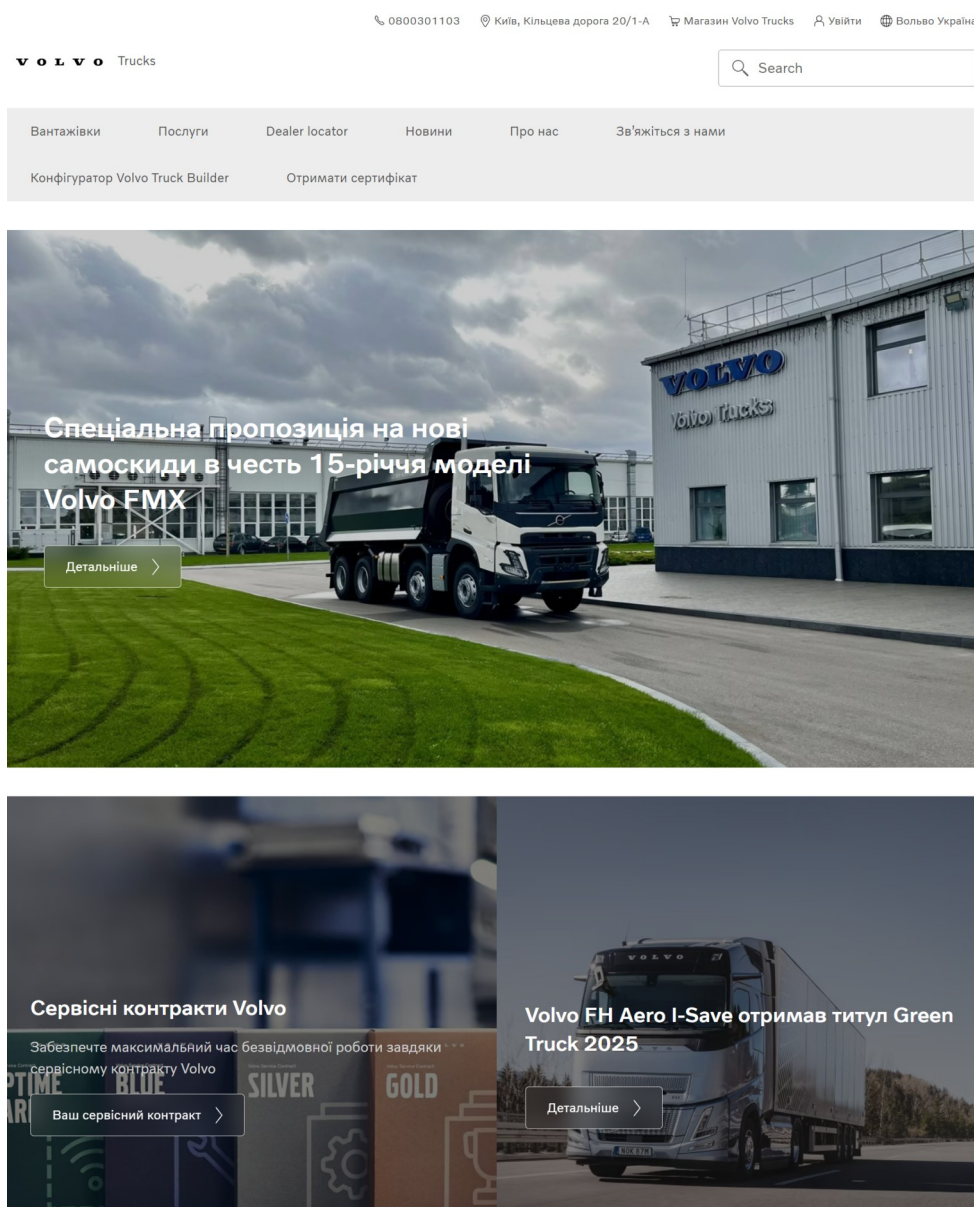


Рисунок 1.2 — Вигляд веб-застосунку

1.2.3 Веб-застосунок DAF

Веб-сайт DAF[5] також є офіційним сайтом компанії. Він має гарний інтерфейс та можливість пошуку автосалонів.

Переваги:

- гарний інтерфейс;
- можливість знайти найближчий автосалон.

Вигляд застосунку наведено на рисунку 1.3 нижче:

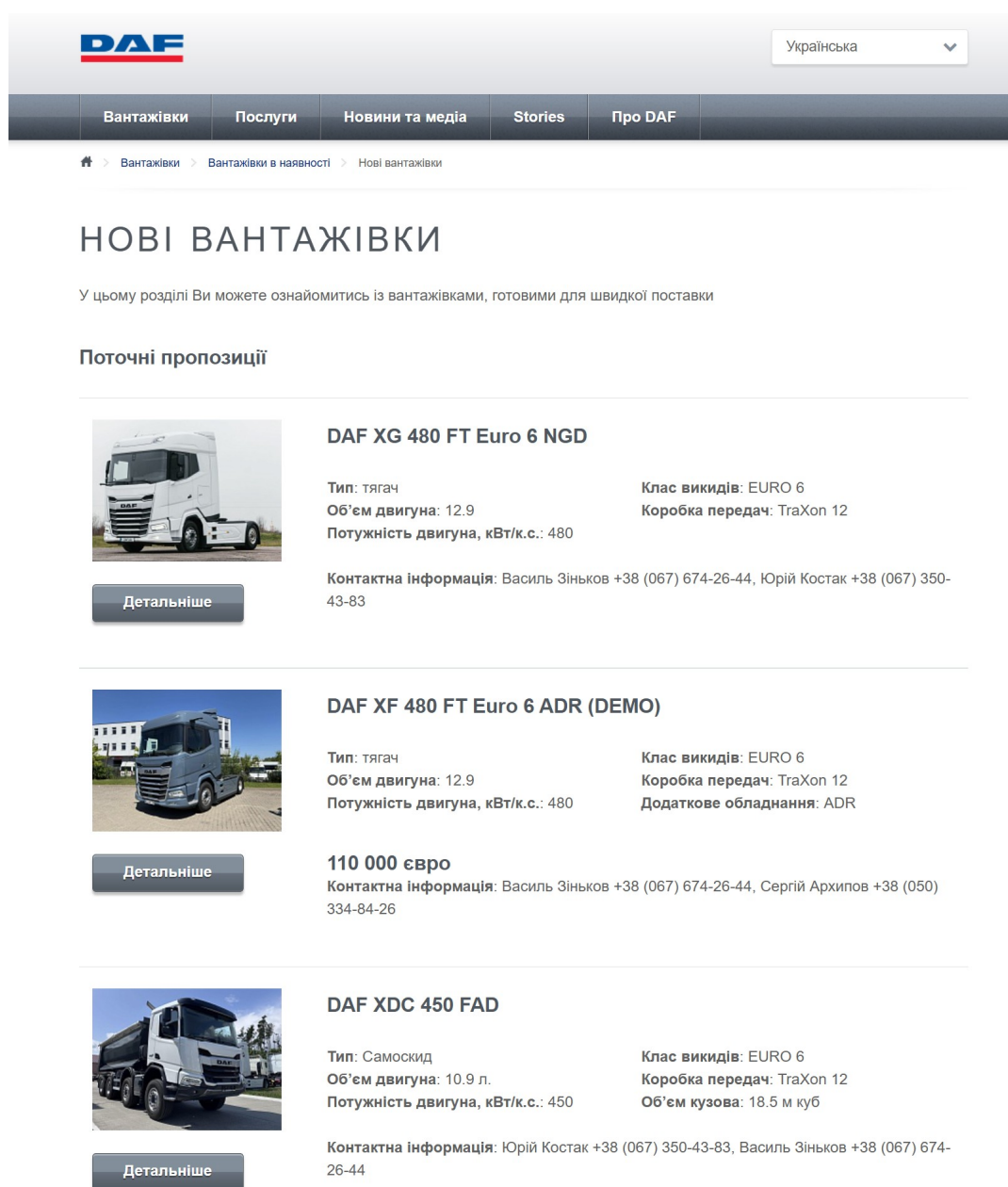


Рисунок 1.3 — Вигляд сторінки продажу

1.2.4 Веб-застосунок RST

Веб-застосунок RST[6] є веб-застосунком продажу та купівлі автомобілів. Він має зрозумілий інтерфейс та численні фільтри.

Переваги:

- зрозумілий інтерфейс;
- доступні фільтри;
- можливість авторизації.

Вигляд застосунку наведено нижче на рисунку 1.4:

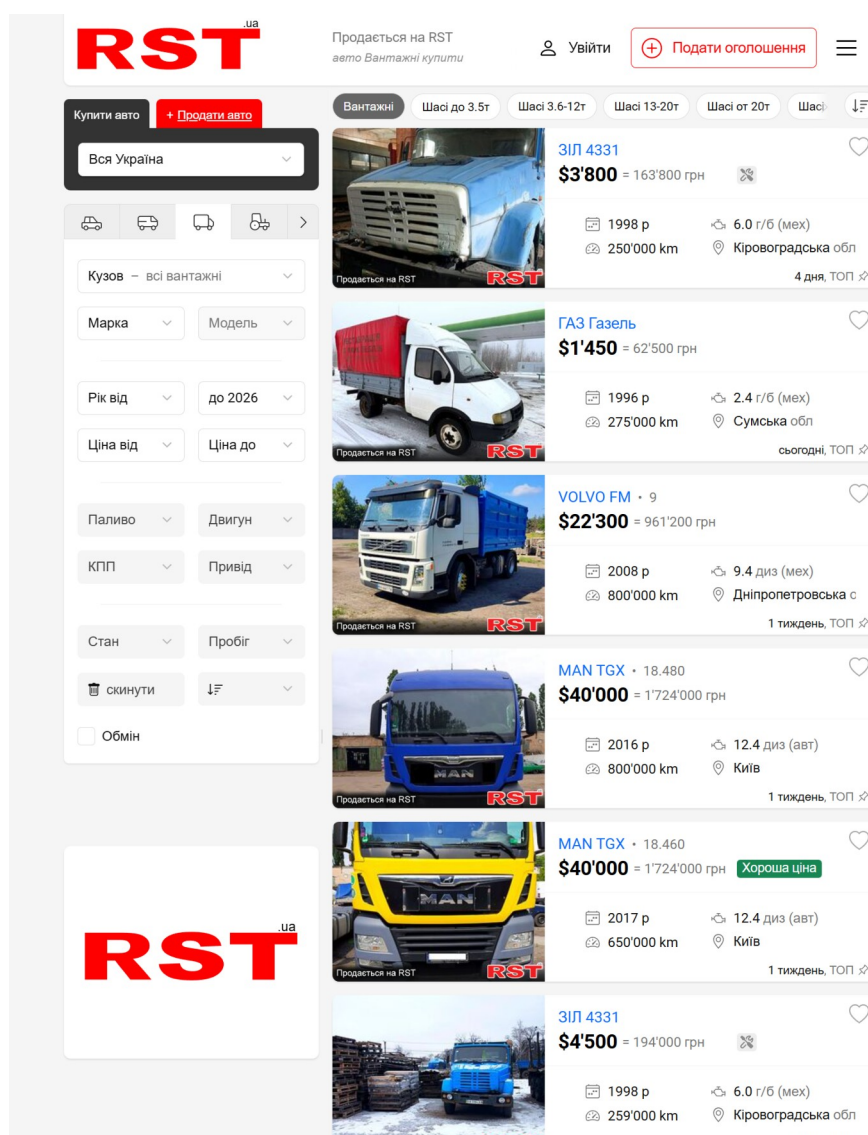


Рисунок 1.4 — Вигляд веб-застосунку

1.2.5 Веб-застосунок OLX

Веб-застосунок OLX[7] містить велику кількість фільтрів, гарний користувацький інтерфейс та можливість написати продавцеві у чат.

Переваги:

- велика кількість фільтрів;
- світлий інтерфейс;
- можливість написати продавцеві у чат.

Вигляд застосунку наведено на рисунку 1.5 нижче:

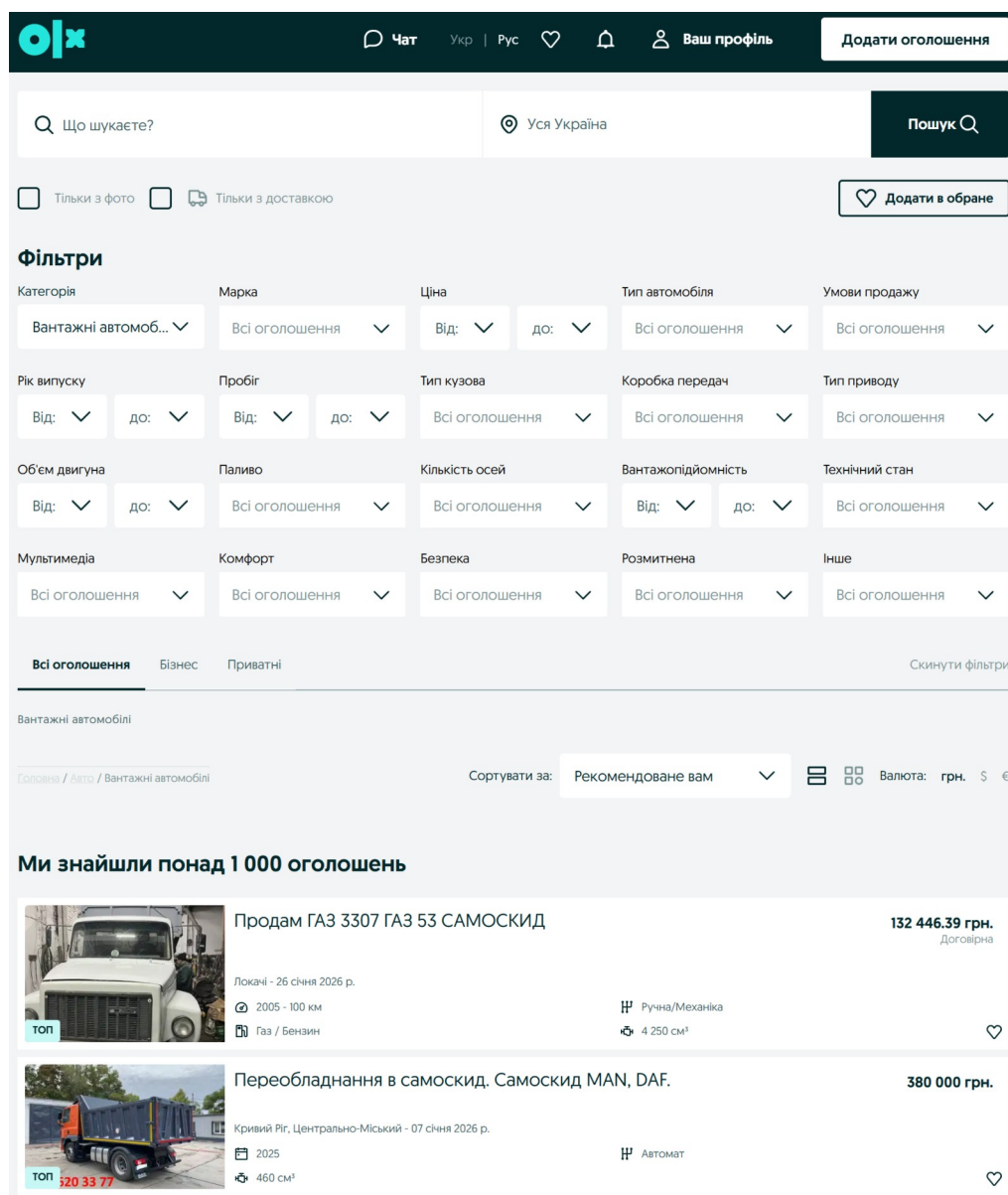


Рисунок 1.5 — Вигляд веб-застосунку

1.3 Порівняння застосунків

Порівняння застосунків наведено на таблиці 1.1 нижче:

Таблиця 1.1 — Порівняльна таблиця застосунків

Критерій	AutoRIA	Volvo	DAF	RST	OLX
Зручність інтерфейсу	+	+	+	-	-
Можливість знайти автосалон	-	+	+	-	-
Можливість авторизації	+	+	-	+	+
Наявність фільтрів	+	-	-	+	+
Можливість продажу власної машини	+	-	-	+	+

В результаті порівняння видно, що застосунок AutoRIA перевершує всі інші аналоги.

1.4 Вимоги до розробки

З огляду аналогів можна винести такі вимоги:

- зручний графічний інтерфейс користувача;
- можливість купівлі вантажівок;
- можливість взяття кредиту для вантажівки.

1.5 Постановка задачі

Таким чином, необхідно зробити веб-сайт зі зручним графічним інтерфейсом користувача, на якому можна буде брати кредити на вантажівки, продавати та купувати вантажівки, здійснювати замовлення на вантажівках.

1.5 Висновки за розділом

Отож, всі розглянуті застосунки містять зручний графічний інтерфейсу користувача та багато корисних функцій для спрощення пошуку вантажівок.

Підставою створення веб-застосунку є збільшення швидкості пошуку вантажних автомобілів для їх подальшої купівлі.

2 РОЗРОБКА АРХІТЕКТУРИ ПРОГРАМИ

2.1 Вступ

Заплановано використати такі технології:

- Python – мова програмування;
- NiceGUI – фул-стак бібліотека розробки графічних інтерфейсів;
- SQLite – файлова база даних.

2.2 Вибір інструментів розробки

2.2.1 Вибір мови програмування

Варіантами мови програмування є:

- Python;
- C#;
- TypeScript.

Порівняння цих мов програмування наведено на таблиці нижче:

Таблиця 2.1 — Порівняння мов програмування

Критерій	Python	TypeScript	C#
Типізація	Динамічна	Статична	Статична
Наявність бібліотек	+	+	-
Підтримка спільноти	+	+	-
Простота синтаксису	++	+	+

Обрано мову програмування Python через її простий синтаксис, динамічну типізацію та підтримку спільноти.

2.2.2 Вибір середовища розробки

Варіантами середовища розробки є:

- Visual Studio Code;

- PyCharm.

Порівняння цих середовищ розробки наведено на таблиці нижче:

Таблиця 2.2 — Порівняння інтегрованих середовищ розробки

Критерій	Visual Studio Code	PyCharm
Простота	+	-
Кількість налаштувань	-	+
Підтримка спільноти	+	-
Наявність плагінів	++	+

Обрано Visual Studio Code через її простий інтерфейс, високу підтримку спільноти та високу кількість плагінів.

2.2.3 Вибір бібліотеки інтерфейсу

Варіантами бібліотеки для графічного інтерфейсу користувача є:

- NiceGUI;
- Tkinter;
- Streamlit.

Порівняння цих бібліотек наведено на таблиці нижче:

Таблиця 2.3 — Порівняння бібліотек для інтерфейсу

Критерій	NiceGUI	Streamlit	Tkinter
Платформа	Web	Web	Desktop
Персоналізація	++	-	+
Швидкість розробки	++	+	+
Зрозумілість документації	++	+	-

Обрано NiceGUI через його високий рівень можливостей персоналізації, швидкість роботи та зрозумілість документації.

2.2.4 Вибір бібліотеки бази даних

Варіантами бібліотеки для бази даних є:

- peewee;
- SQLAlchemy;
- sqlite3.

Їх порівняння наведено на таблиці нижче:

Таблиця 2.4 — Порівняння бібліотек бази даних

Критерій	peewee	SQLAlchemy	sqlite3
Простота налаштування	++	-	+
Простота синтаксису	++	-	+
Швидкість роботи	++	+	+

Обрано бібліотеку peewee через її швидкість роботи, простоту налаштування та зрозумілість синтаксису.

2.3 Вибір і проєктування бази даних

Для цієї програми можна обрати такі бази даних:

- SQLite;
- PostgreSQL;
- MySQL.

Їх порівняння наведено на таблиці 2. нижче:

Таблиця 2.5 — Порівняння баз даних

Критерій	SQLite	PostgreSQL	MySQL
Масштабованість	-	++	+
Швидкість	++	+	-
Простота налаштування	++	-	+

Обрано базу даних SQLite через її простоту налаштування, швидкість роботи та файлове базування.

Далі було спроектовано таблиці бази даних. Їх наведено нижче:

Таблиця 2.6 — Таблиця Truck

Властивість	Тип даних	Опис
Name	String	Назва вантажівки
Category	String	Категорія: Бус Вантажівка Всюдихід
Price	Integer	Ціна вантажівки
Picture	String	Назва картинки вантажівки

Таблиця 2.7 — Таблиця Loan

Властивість	Тип даних	Опис
Amount	Integer	Розмір кредиту в грошах
Duration	Integer	Тривалість кредиту (місяці)
Bank	String	Назва банку, який видає кредит

Таблиця 2.8 — Таблиця Order

Властивість	Тип даних	Опис
Price	Integer	Вартість замовлення
Location	String	Назва населеного пункту, куди замовлення везти
Coordinates	Pair<float, float>	Координати населеного пункту

Після цього було створено схеми бази даних. Їх наведено нижче на рисунках:

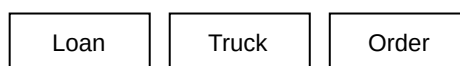


Рисунок 2.1 — Концептуальна схема

Loan	Truck	Order
Amoun: int	Name: str	Price: int
Duration: int	Category: str	Location: str
Bank: str	Price: int	Coordinates: pair<float, float>
	Picture: str	

Рисунок 2.2 — Логічна схема

Truck	Loan	Order
name = Char	amount = Integer	price = Integer
category = Char	duration = Integer	location = Char
price = Integer	bank = Char	coordinates = Char
picture = Char		

Рисунок 2.3 — Фізична схема

2.4 Висновки за розділом

Отож, у цьому розділі було обрано мову програмування, інтегроване середовище розробки, бібліотеку графічного інтерфейсу користувача, бібліотеку взаємодії з базою даних, обрано та розроблено базу даних.

3 ПРОГРАМНА РЕАЛІЗАЦІЯ

3.1 Вступ

Програму створено так:

- клієнтська частина – інтерфейс через NiceGUI;
- серверна частина – моделі даних через бібліотеку peewee, під'єднання до бази даних SQLite.

3.2 Реалізація клієнтської частини

Клієнтська частина розташована у файлі main.py. Вона містить такі частини:

1. змінні стану State: клас з різними змінними, які використовуються у програмі під час роботи. Включає такі змінні:

- trucks: всі вантажівки;
- loans: всі кредити;
- orders: всі замовлення;
- owned_trucks: власні автомобілі;
- selected_truck: обраний автомобіль;
- taken_loans: взяті кредити;
- money: гроші користувача.

2. клас констант Const: містить константні змінні, які можуть використовуватися у програмі під час роботи. Містить такі змінні:

- current_folder: шлях до поточної папки.

3. функція convert_months_to_years:

- конвертувати кількість місяців у рядок, який містить кількість років та місяців, якщо доступні, або у кількість років, або у кількість місяців.

4. функція оновлення грошей update_money:

- оновити мітку грошей значенням зі змінної стану.

5. функція оновлення обраної вантажівки update_selected_truck:

- оновити мітку обраної вантажівки із назвою автомобіля.

6. функція виконання замовлення `perform_order`:

- додати до змінної стану грошей вартість замовлення;
- оновити мітку грошей та показує місто замовлення на мапі.

7. функція обрання вантажівки `select_truck`:

- встановити змінну стану обраної вантажівки у надану вантажівку;
- оновити мітку.

8. функція придбання вантажівки `buy_truck`:

- відняти зі змінної стану грошей вартість вантажівки;
- оновити мітку грошей;
- до змінної стану придбаних вантажівок додати надану;
- оновити сторінку;
- обрати надану вантажівку;
- вивести повідомлення користувачеві.

9. функція продажу вантажівки `sell_truck`:

- як обрану вантажівку поставити значення нуль;
- оновити мітку обраної вантажівки;
- зі змінної стану придбаних вантажівок видалити надану;
- оновити сторінку;
- повідомити користувача про успішний продаж автомобіля;
- до змінної стану грошей додати вартість вантажівки;
- оновити мітку грошей.

10. функція взяття кредиту `take_loan`:

- до змінної стану грошей додає вартість кредиту;
- до змінної стану взятих кредитів додати наданий;
- повідомити користувача про успішно взятий кредит;
- оновити сторінку.

11. функція сплати кредиту `pay_loan`:

- перевірити чи у змінній стану менше грошей за вартість кредиту;
- якщо так, повідомити про це користувачеві та завершити роботу;

- від змінної стану грошей відняти вартість кредиту;
- від змінної стану взятих кредитів прибрати наданий;
- оновити гроші;
- оновити сторінку;
- повідомити користувача про успішну сплату кредиту.

12. сторінка кредитів `loans_view`:

- якщо немає взятих кредитів у змінній стану кредитів, вивести повідомлення про це;
- для кожного кредиту у змінній стану взятих кредитів показати картку, в якій показати назву кредиту, його тривалість, його вартість та кнопку сплати.

13. сторінка придбаних вантажівок `owned_trucks_view`:

- якщо немає придбаних вантажівок у змінній стану, вивести про це повідомлення;
- для кожної вантажівки у змінній стану придбаних вантажівок вивести картку, в якій показати зображення вантажівки, її назву, категорію та кнопки обрання та продажу.

14. сторінка автосалону `store_view`:

- для кожної вантажівки у змінній стану всіх вантажівок вивести картку, на якій показати картинку вантажівки, її назву, категорію, вартість та кнопку купівлі.

15. сторінка банку `bank_view`:

- для кожного кредиту зі змінної стану усіх кредитів вивести картку, на якій показати назву кредиту, його тривалість, вартість та кнопку взяття.

16. сторінка замовлень `orders_view`:

- показати мапу;
- для кожного замовлення зі змінної стану усіх замовлень вивести картку, на якій показати локацію замовлення, його вартість та кнопку його виконання.

Далі змінна стану вантажівок, кредитів та замовлень заповнюється через функції з серверної частини.

Після цього створюються різні сторінки.

3.3 Реалізація серверної частини

Серверна частина розташована у файлі `data.py`. Вона містить такі частини:

1. інтерфейс назв сторінок `TabName`:
 - містить назви усіх сторінок застосунку.
2. інтерфейс категорій вантажівок `TruckCategory`:
 - містить назви категорій вантажівок.
3. інтерфейс назв банків `BankName`:
 - містить назви усіх банків.
4. клас вантажівки `ITruck`, містить такі змінні:
 - `name (str)`: назва вантажівки;
 - `category (TruckCategory)`: категорія вантажівки;
 - `price (int)`: вартість вантажівки;
 - `picture (str)`: назва файлу картинки.
5. клас кредиту `ILoan`, містить такі змінні:
 - `amount (int)`: вартість кредиту;
 - `duration (int)`: тривалість кредиту у місяцях;
 - `bank (BankName)`: банк, який дає кредит.
6. клас замовлення `IOrder`, містить такі змінні:
 - `price (int)`: вартість замовлення;
 - `location (str)`: місто, куди буде прямувати замовлення;
 - `coordinates (tuple<float, float>)`: координати міста на мапі.

Після цього програма доєднується до бази даних `SQLite`.

Далі серверна частина містить визначення моделей бази даних, деталі нижче:

1. модель вантажівки Truck, містить такі поля:

- name – CharField;
- category – CharField;
- price – IntegerField;
- picture – CharField.

2. модель кредиту Loan, містить такі поля:

- amount – IntegerField;
- duration – IntegerField;
- bank – CharField.

3. модель замовлення Order, містить такі поля:

- price – IntegerField;
- location – CharField;
- coordinates – CharField.

Діаграми класів моделей наведено нижче на рисунках 3.1-3.3:

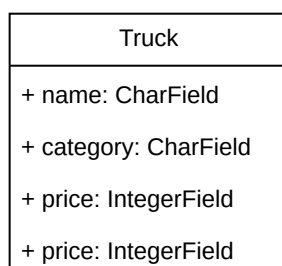


Рисунок 3.1 — Діаграма класу моделі вантажівки Truck

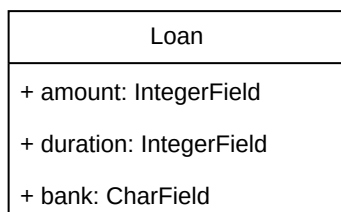


Рисунок 3.2 — Діаграма класу моделі кредиту Loan

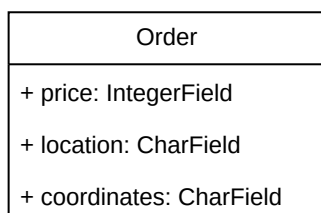


Рисунок 3.3 — Діаграма класу моделі замовлення Order

Після цього серверна частина містить функції завантаження даних:

1. функція завантаження вантажівок load_trucks:
 - завантажує з бази даних та повертає усі вантажівки.
2. функція завантаження кредитів load_loans:
 - завантажує з бази даних та повертає усі кредити.
3. функція завантаження замовлень load_orders:
 - завантажує з бази даних та повертає усі замовлення.

Нижче наведено ER-діаграму моделей бази даних:

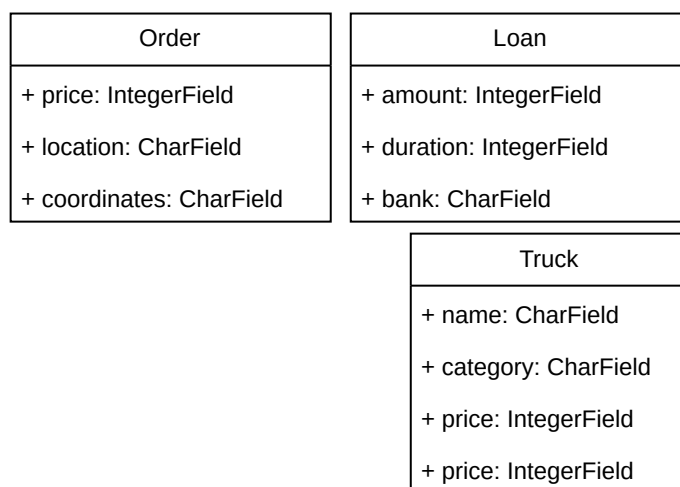


Рисунок 3.4 - ER-діаграма

Діаграма не містить зв'язків, бо у моделях бази даних немає чужих ключів.

3.4 Взаємодія клієнта та сервера

Серверна частина доєднується до бази даних через бібліотеку `reewee`, обрану раніше. Спочатку оголошується змінна назви бази даної та її об'єкт. Після цього створюється базова модель — клас `BaseModel`, від якого спадкують усі інші класи моделей. В цьому класі оголошується змінна `database` і встановлюється у раніше створений об'єкт.

Клієнтська частина взаємодіє з серверною через вище описані функції завантаження даних. Клієнт на початку роботи завантажує дані з бази даних через ці функції. Після цього вони призначаються до змінної стану та використовуються у програмі через неї.

Функції завантаження даних використовують метод `select`, яка обирає усі доступні об'єкти наданої моделі.[8]

Нижче наведено таблицю, яка показує які сторінки використовують які з моделей даних:

Таблиця 3.1 — Які сторінки використовуються які з моделей

<i>Модель даних</i>	Кредити	Гараж	Автосалон	Банк	Замовлення
Truck		+	+		
Loan	+			+	
Order					+

3.5 Висновки за розділом

Отож, програму реалізовано, вона складається з двох частин — клієнтської та серверної.

4 ТЕСТУВАННЯ ПРОГРАМИ

4.1 Вступ

Програму заплановано протестувати на системі Windows 10 (64-bit) на кількох браузерях.

4.2 Призначення програми

Програма призначена для купівлі та продажу вантажних автомобілів. Вона дозволяє виконувати замовлення на вантажівках, брати кредити та сплачувати їх.

4.3 Процес тестування

Тестування проводилося у браузері Microsoft Edge 64-біт за тест-кейсами, описаними нижче.

4.3.1 Тест кейс один — Здійснення замовлень

Кроки виконання тест-кейсу:

1. зайти на сайт;
2. перейти до сторінки Банку;
3. взяти кредит на 8 000 гривень;
4. перейти на сторінку Автосалону;
5. придбати Renault Master;
6. перейти на сторінку Замовлень;
7. виконати замовлення у Лондоні;
8. виконати замовлення у Варшаві.

Процес виконання тест-кейсу один наведено нижче на таблиці 4.1:

Таблиця 4.1 — Результати тест-кейсу один

№	Крок	Успіх?
1	Зайти на сайт	+
2	Перейти до сторінки Банку	+
3	Взяти кредит на 8 000 гривень	+
4	Перейти на сторінку Автосалону	+
5	Придбати Renault Master	+
6	Перейти на сторінку Замовлень	+
7	Виконати замовлення у Лондоні	+
8	Виконати замовлення у Варшаві	+

Тестування за тест-кейсом один пройшло успішно.

4.3.2 Тест кейс два — Продаж вантажівки

Кроки виконання тест-кейсу:

1. зайти на сайт;
2. перейти до сторінки Банку;
3. взяти кредит на 6 000 гривень;
4. перейти на сторінку Автосалону;
5. придбати Mercedes Sprinter Offroad;
6. перейти на сторінку Гаражу;
7. продати Mercedes Sprinter Offroad.

Процес виконання тест-кейсу два наведено нижче на таблиці 4.2:

Таблиця 4.2 — Результати тест-кейсу два

№	Крок	Успіх?
1	Зайти на сайт	+
2	Перейти до сторінки Банку	+
3	Взяти кредит на 6 000 гривень	+
4	Перейти на сторінку Автосалону	+
5	Придбати Mercedes Sprinter Offroad	+
6	Перейти на сторінку Гаражу	+

№	Крок	Успіх?
7	Продати Mercedes Sprinter Offroad	+

Тестування за тест-кейсом два пройшло успішно.

4.3.3 Тест кейс три — Сплачення кредитів

Кроки виконання тест-кейсу:

1. зайти на сайт;
2. перейти до сторінки Банку;
3. взяти кредит на 12 000 гривень;
4. взяти кредит на 3 000 гривень;
5. перейти на сторінку Автосалону;
6. придбати Volvo FH 750;
7. перейти на сторінку Замовлень;
8. виконати замовлення у Харкові;
9. перейти на сторінку Кредитів;
10. сплатити кредит на 12 000 гривень;
11. сплатити кредит на 3 000 гривень;

Процес виконання тест-кейсу три наведено нижче на таблиці 4.3:

Таблиця 4.3 — Результати тест-кейсу три

№	Крок	Успіх?
1	Зайти на сайт	+
2	Перейти до сторінки Банку	+
3	Взяти кредит на 12 000 гривень	+
4	Взяти кредит на 3 000 гривень	+
5	Перейти на сторінку Автосалону	+
6	Придбати Volvo FH 750	+
7	Перейти на сторінку Замовлень	+
8	Виконати замовлення у Харкові	+
9	Перейти на сторінку Кредитів	+

№	Крок	Успіх?
10	Сплатити кредит на 12 000 гривень	+
11	Сплатити кредит на 3 000 гривень	+

Тестування за тест-кейсом три пройшло успішно.

4.4 Висновок за розділом

Отже, програму було успішно протестовано. Вона працює коректно.

5 КЕРІВНИЦТВО КОРИСТУВАЧА

5.1 Необхідні умови для виконання

Для виконання програми потрібно:

1. завантажити архів програми собі на систему;
2. розархівувати код програми;
3. завантажити та встановити Python останньої версії.

Аби запустити програму потрібно:

1. з теки програми ввести команду `.venv\Scripts\activate.bat`, далі ввести `python main.py`;
2. перейти до сторінки <http://localhost:8080>.

5.2 Звертання до програми

Програма складається з п'яти сторінок. На початку показується сторінка автосалону.

Опис сторінок наведено на таблиці нижче:

Таблиця 5.1 — Опис кожної сторінки

Сторінка	Опис
Кредити	Взяті кредити, де їх можна сплатити
Гараж	Куплені вантажівки, де їх можна переглянути
Автосалон	Магазин вантажівок, де їх можна купити
Банк	Сторінка кредитів, де їх можна взяти
Замовлення	Сторінка замовлень, де їх можна виконати

Верхнє меню містить значки грошей та обраної вантажівки.

Значок грошей позначено символом , а значок вантажівки символом .

Значок грошей показує поточний баланс користувача. Значок вантажівки показує обрану вантажівку, на якій здійснюються замовлення.

5.2.1 Сторінка кредитів

Сторінка кредитів виглядає наступним чином:

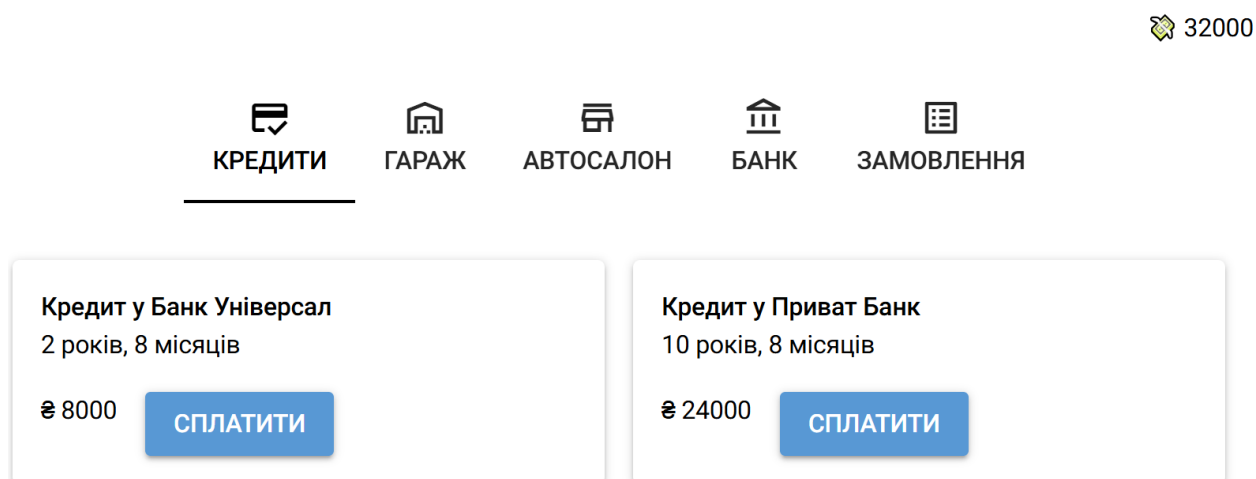


Рисунок 5.1 — Вигляд сторінки кредитів

На цій сторінці відображаються взяті кредити. Їх можна оплатити, натискаючи на кнопку Сплатити під відповідним кредитом.

Коли кредити відсутні, сторінка виводить наступне повідомлення:



Немає взятих кредитів.

Рисунок 5.2 — Сторінка кредитів коли жоден не взятий

5.2.2 Сторінка гаражу

Сторінка вантажівок виглядає так:

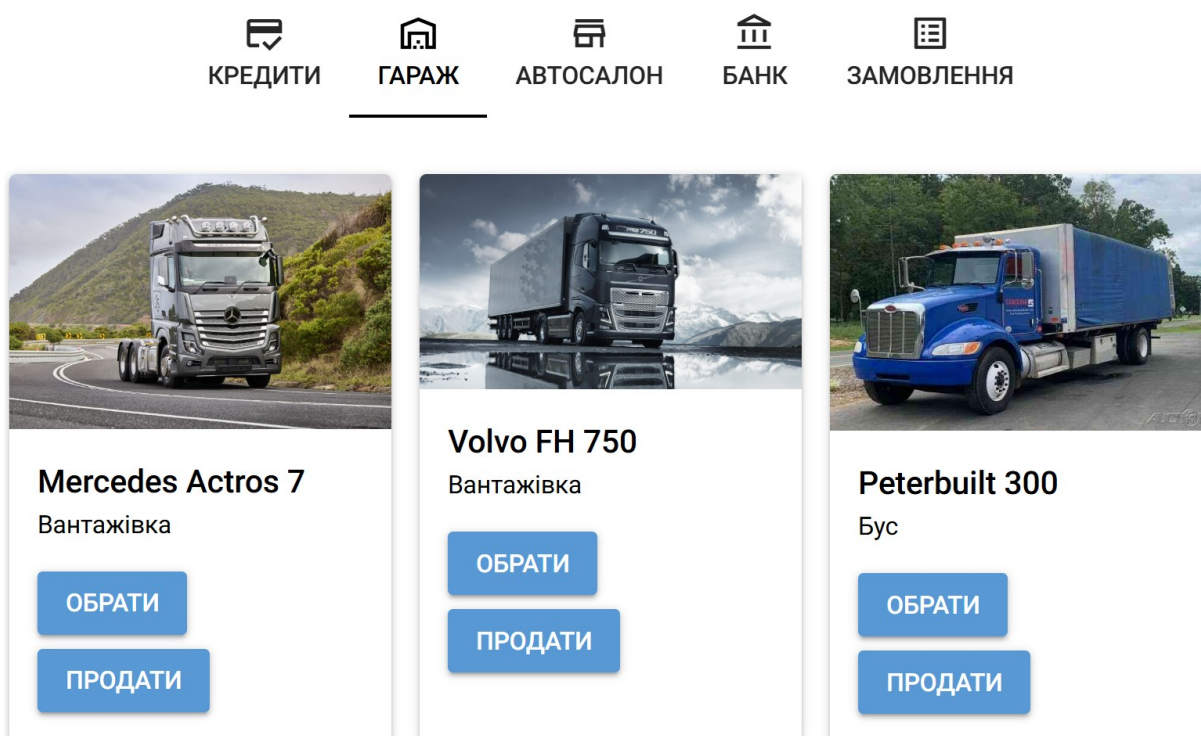


Рисунок 5.3 — Вигляд сторінки вантажівок

Вантажівку можна обрати, для цього потрібно натиснути кнопку Обрати під відповідним автомобілем.

Кожен автомобіль можна продати натисканням кнопки Продати під відповідним автомобілем.

Коли жодної вантажівки не куплено, показується наступне повідомлення:



Немає придбаних вантажівок.

Рисунок 5.4 — Вигляд сторінки вантажівок, коли жодної не куплено

5.2.3 Сторінка автосалону

Сторінка автосалону має такий вигляд:

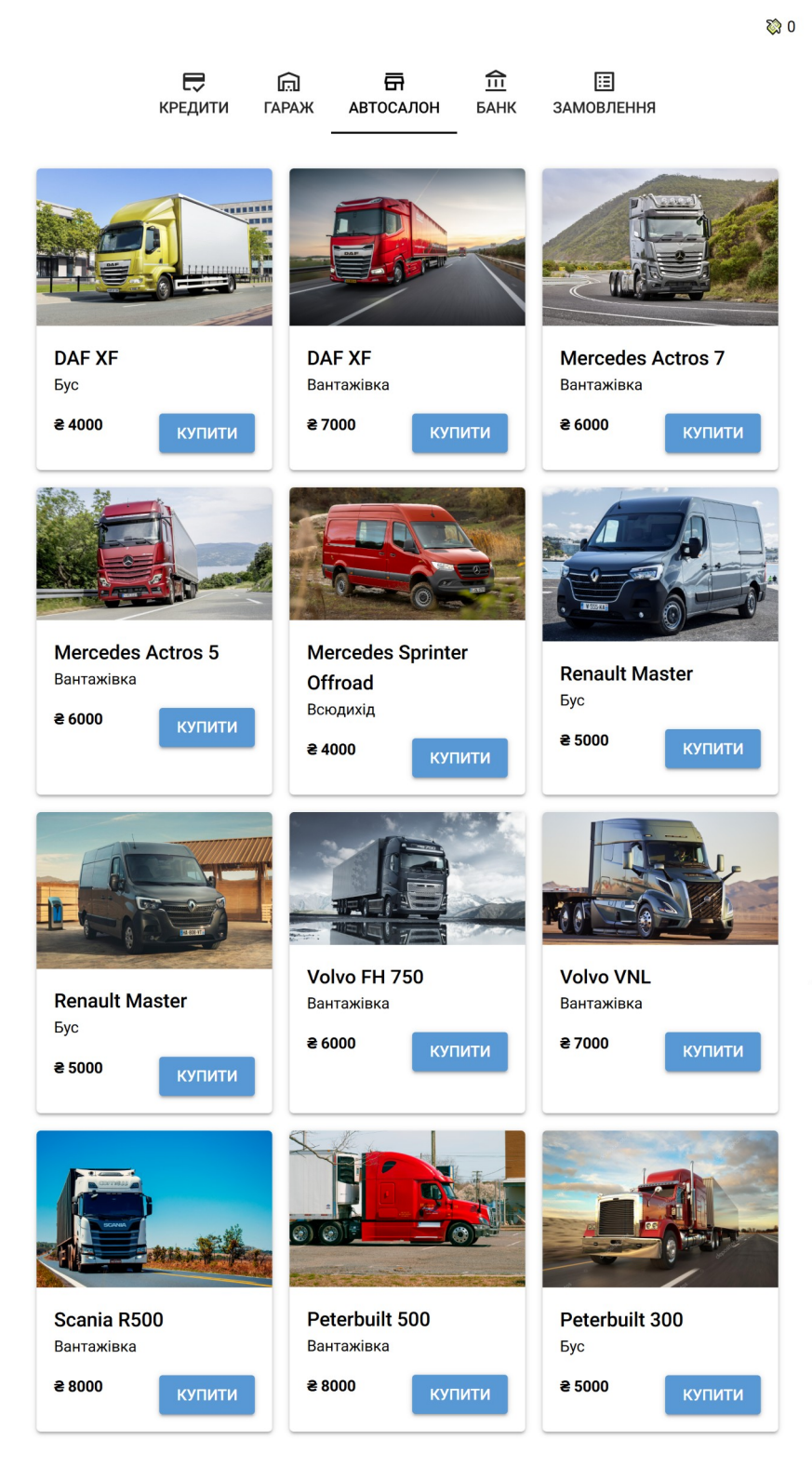


Рисунок 5.5 — Вигляд сторінки автосалону

На сторінці можна купити будь-яку вантажівку, натиснувши під нею кнопку Купити.

5.2.4 Сторінка банку

Вигляд сторінки банку наведено нижче на рисунку:

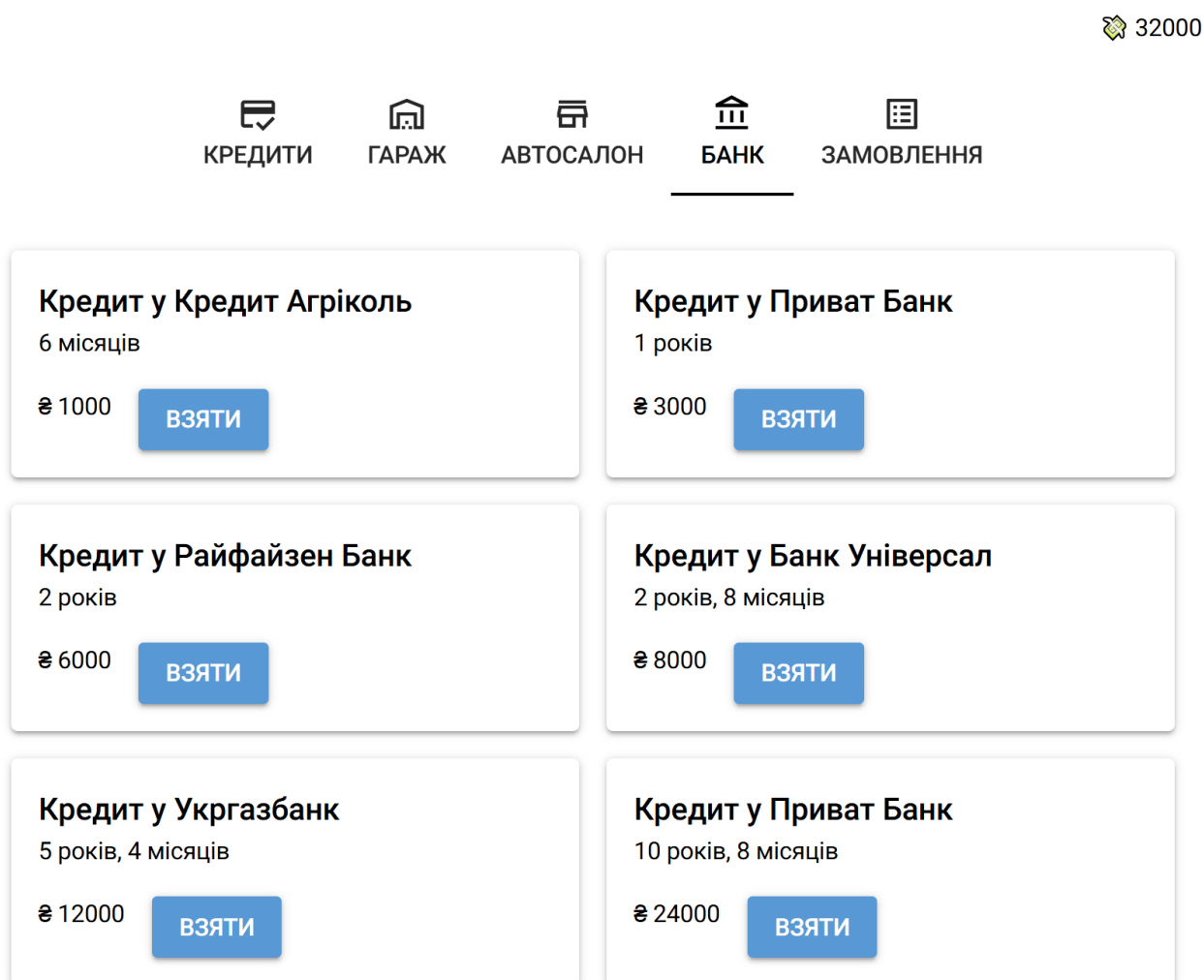


Рисунок 5.6 — Вигляд сторінки

Користувач може брати будь-яку кількість кожного з кредитів, натиснувши кнопку Взяти під відповідним кредитом.

5.2.5 Сторінка замовлень

Сторінка замовлень має наступний вигляд:

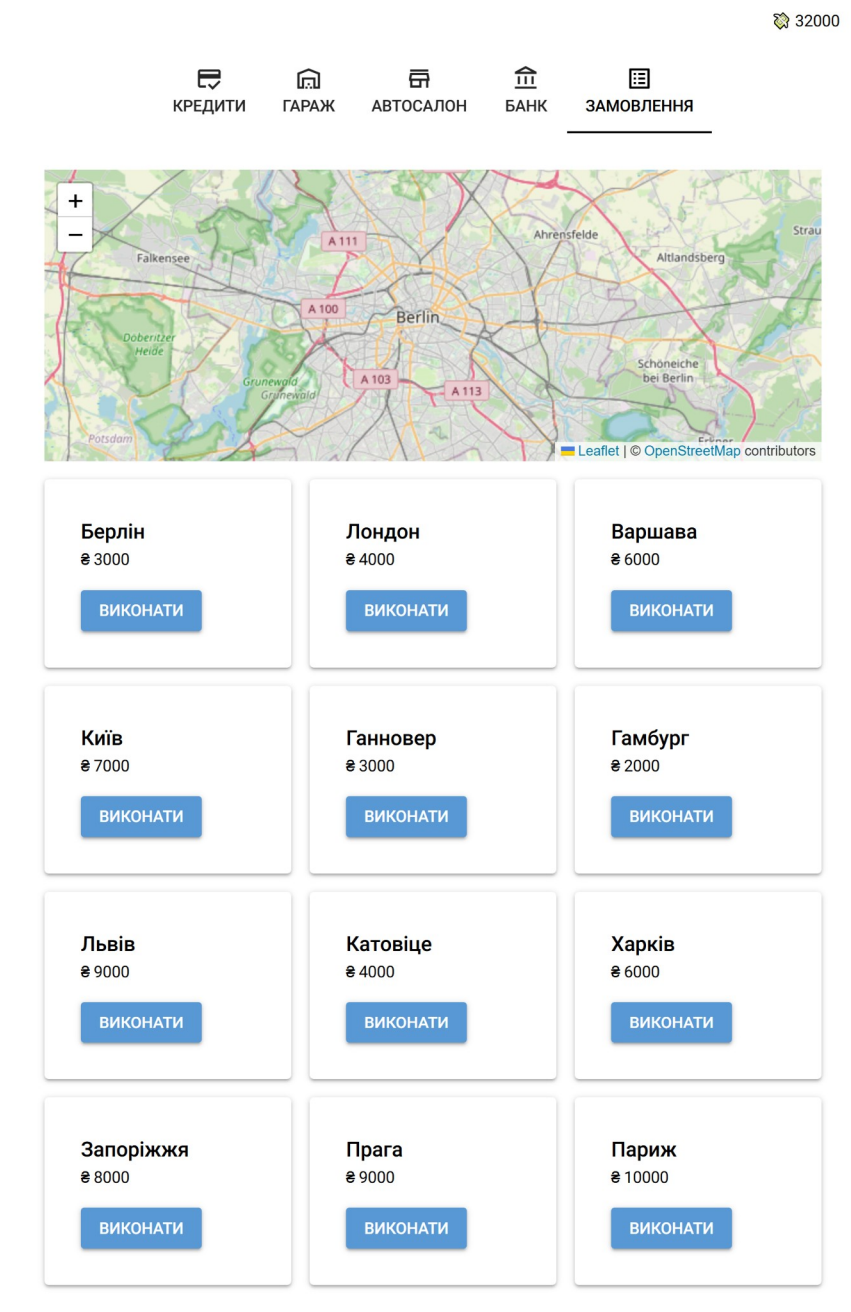


Рисунок 5.7 — Вигляд сторінки замовлень

На мапі буде показано місто, до якого здійснюється замовлення. При натисканні кнопки Виконати буде показано відповідне місто.

ВИСНОВКИ

У результаті виконання дипломної роботи бакалавра було розроблено програмне забезпечення для купівлі вантажних автомобілів, яке дозволяє ефективно обирати вантажівки для продажу та виконувати замовлення на них.

Проведено аналіз існуючих технологій для пошуку вантажівок, що дозволило обрати оптимальний набір функцій: купівля вантажівок, їх продаж, виконання замовлень та взяття кредитів.

ПЕРЕЛІК ДЖЕРЕЛ ПОСИЛАННЯ

1. Вантажні перевезення Запоріжжя: Ціна на вантажні перевезення в Запоріжжі. *Транспортна компанія в Києві - Сайт вантажних перевезень.*

URL: <https://avrora-trans.com/ua/services/ukraine/gruzoperevozki-zaporozhe>

(дата звернення: 29.04.2026).

2. Тренди та виклики українського ринку eCommerce у 2024 році | UAATEAM. *UAATEAM | Агенція діджитал маркетингу для малого та середнього бізнесу.*

URL: <https://uaateam.agency/blog/trendy-ta-vyklyky-ukrainskogo-rynku-ecommerce/> (дата звернення: 29.04.2026).

3. AUTO.RIA - Базар авто №1: автосалони, продаж авто б/в і нових. Автопошук по 377 оголошень України. AUTO.RIA Базар авто. *AUTO.RIA.*

URL: <https://auto.ria.com/uk/search/?>

[search_type=1&category=6&has_video_messages=1&abroad=0&customs_cleared=1&page=0&limit=20](https://auto.ria.com/uk/search/?search_type=1&category=6&has_video_messages=1&abroad=0&customs_cleared=1&page=0&limit=20) (дата звернення:

29.04.2026).

4. Вантажівки Volvo Trucks - UK. *Вантажівки Volvo Trucks.*

URL: <https://www.volvotrucks.com.ua/uk-ua/> (дата звернення: 28.04.2026).

5. Нові вантажівки | DAF Trucks Ukraine. *DAF Trucks Ukraine! Купити, дилери, ціни, СТО, запасні частини, сервіс - DAF Trucks Ukraine.*

URL: <https://daf.ua/trucks/trucks-in-stock/new-trucks/> (дата звернення:

28.04.2026).

6. Продається на RST | авто Вантажні | Купити Вантажні. *RST Продаж вантажівок.*

URL: <https://rst.ua/ukr/oldcars/?k=2&task=newresults>. (дата

звернення: 28.01.2026)

7. Оголошення OLX.ua: сервіс оголошень України – купівля/продаж бу та нових товарів, різноманітні послуги на сайті OLX.ua. *Оголошення OLX.ua: сервіс оголошень України – купівля/продаж бу та нових товарів, різноманітні послуги на сайті OLX.ua.*

URL: <https://www.olx.ua/uk/transport/gruzovye-avtomobili/?currency=UAH> (дата звернення: 28.04.2026).

8. Querying – peewee 4.0.5 documentation. *peewee – peewee 4.0.5 documentation*.

URL: <https://docs.peewee-orm.com/en/latest/peewee/querying.html> (дата звернення: 28.04.2026).

ДОДАТОК А
Технічне завдання

Вступ

Програма має назву «Застосунок продажу вантажівок».

Програма призначена для купівлі вантажівок та їх використання для здійснення завдань.

Вона може бути використана будь-яким користувачем, якому цікаво працювати на вантажівці.

Підстави для розробки

Підставою розробки є створення застосунку, який має полегшити купівлю вантажних автомобілів.

Призначення розробки

Програма має дозволяти купівлю вантажівок, здійснення замовлень на них.

Вимоги до програми

Основними вимогами до програми є:

- платформа — веб-браузер;
- світлий графічний інтерфейс користувача.

Вимоги до функціоналу

Програма має виконувати такі функції:

- купівля автомобілів;
- взяття кредитів;
- продаж автомобілів.

Вимоги до технічних та програмних засобів

Програма має працювати у веб-браузері. Вона має бути доступна як для мобільних, так і для десктопних платформ.

- процесор: x64 / x86 / ARM;
- оперативна пам'ять: 1 ГБ;
- накопичувач: 5 ГБ вільного місця;
- операційна система: Android, iOS, macOS, Linux, Windows.

Програмні засоби можуть змінюватися в процесі розробки, але заплановано використати такі:

- мова програмування: Python;
- фреймворк інтерфейсу: NiceGUI;
- база даних: SQLite.

Вимоги до програмної документації

Програмна документація має бути реалізована у:

- пояснювальній записці;
- технічному завданні.

Код програми має бути написаний із зрозумілими назвами змінних/класів/методів і може включати коментарі.

ДОДАТОК Б

Текст програми

Текст головного файлу main.py

```

from typing import Optional
from nicegui import ui
import os

from data import (
    ILoan,
    IOrder,
    TabName,
    ITruck,
    load_loans,
    load_orders,
    load_trucks,
)

class State:
    # Всі вантажівки
    trucks: list[ITruck] = list()
    # Всі кредити
    loans: list[ILoan] = list()
    # Всі замовлення
    orders: list[IOrder] = list()

    # Власні вантажівки
    owned_trucks: list[ITruck] = list()
    # Обрана вантажівка
    selected_truck: Optional[ITruck] = None
    # Взяті кредити
    taken_loans: list[ILoan] = list()
    # Гроші користувача
    money: int = 0

class Const:
    CURRENT_FOLDER = os.path.dirname(os.path.abspath(__file__))

# --- ПОМІЧНИКИ ---

def convert_months_to_years(duration_in_months: int) -> str:
    MONTHS_IN_A_YEAR = 12

    years = duration_in_months // MONTHS_IN_A_YEAR
    months = duration_in_months % MONTHS_IN_A_YEAR

    return (
        # Повернути роки та місяці
        f"{years} років, {months} місяців"
        # Якщо надано обидва
        if months and years
        # Інакше == роки
        else f"{years} років"
        # Якщо їх надано
        if years
        # Інакше == місяці
        else f"{months} місяців"
    )

```

```

    )

# --- ОНОВЛЕННЯ ---

def update_money():
    money_label.set_text(f"💰 {State.money}")
    money_label.update()

def update_selected_truck():
    selected_truck_label.set_text(
        # Встановити назву обраної вантажівки, якщо вона обрана. Інакше ==
        # нуль
        f"🚚 {State.selected_truck.name}" if State.selected_truck else ""
    )
    selected_truck_label.update()

# --- ЗАМОВЛЕННЯ ---

def perform_order(order: IOrder, map):
    # Якщо вантажівки не обрано
    if State.selected_truck is None:
        # Повідомити користувача про це
        ui.notify("Немає обраної вантажівки.", close_button="Добре")
        # Завершити роботу
        return

    # Додати до грошей користувача ціну замовлення
    State.money += order.price
    # Оновити лейбл
    update_money()

    # На мапі показати місце, куди йде замовлення
    map.set_center(order.coordinates)
    map.update()

# --- ВАНТАЖІВКИ ---

def select_truck(truck: ITruck):
    # Встановити обрану вантажівку у надану
    State.selected_truck = truck
    update_selected_truck()

def buy_truck(truck: ITruck):
    # Якщо користувацьких грошей менше за вартість вантажівки
    if State.money < truck.price:
        # Повідомити про це користувача
        ui.notify("Недостатньо коштів.", close_button="Добре")
        # Завершити роботу
        return

    # Відняти від грошей користувача ціну вантажівки
    State.money -= truck.price
    update_money()

```

```

# До власних вантажівок додати куплену
State.owned_trucks.append(truck)
owned_trucks_view.refresh()

# Встановити нову вантажівку як обрану
select_truck(truck)
# Повідомити користувача про успішну купівлю
ui.notify(f"Куплено {truck.category.lower()} під назвою
{truck.name}!")

def sell_truck(truck: ITruck):
    # Обнулити обрану вантажівку
    if State.selected_truck == truck:
        State.selected_truck = None
        update_selected_truck()

    # Прибрати вантажівку з придбаних
    State.owned_trucks.remove(truck)
    owned_trucks_view.refresh()
    # Повідомити користувача про успішний продаж
    ui.notify(f"Продано {truck.category.lower()} під назвою
{truck.name}!")

    # Додати до грошей користувача ціну вантажівки
    State.money += truck.price
    update_money()

# --- КРЕДИТИ ---

def take_loan(loan: ILoan):
    # Додати до грошей користувача суму кредиту
    State.money += loan.amount
    # До взятих кредитів додати наданий
    State.taken_loans.append(loan)

    update_money()
    # Повідомити користувача про успішне взяття кредиту
    ui.notify(f"Взято кредит на € {loan.amount} від {loan.bank}")
    loans_view.refresh()

def pay_loan(loan: ILoan):
    # Якщо грошей у користувача менше за суму кредиту
    if State.money < loan.amount:
        # Повідомити про це користувача
        ui.notify("Недостатньо коштів для сплати.", close_button="Добре")
        # Завершити роботу
        return

    # Від грошей користувача відняти суму кредиту
    State.money -= loan.amount
    # Від взятих кредитів прибрати наданий
    State.taken_loans.remove(loan)

    update_money()
    bank_view.refresh()
    # Повідомити користувача про успішне сплатення кредиту

```

```

        ui.notify(f"Сплачено кредит на € {loan.amount} від {loan.bank}")

# --- ПРЕДСТАВЛЕННЯ ІНТЕРФЕЙСУ ---

@ui.refreshable
def loans_view():
    # Якщо немає взятих кредитів
    if not State.taken_loans:
        # Повідомити про це користувача
        ui.label("Немає взятих кредитів.")
        return
    with ui.grid(columns=2).classes("w-full"):
        # Для кожного взятого кредиту
        for loan in State.taken_loans:
            # Зробити картку
            with ui.card().tight():
                with ui.card_section():
                    # Назва кредиту
                    ui.label(f"Кредит у {loan.bank}").classes("font-
medium")

                    # Тривалість кредиту
                    ui.label(f"на
{convert_months_to_years(loan.duration)}")
                with ui.row().classes("mt-4"):
                    # Сума кредиту
                    ui.label(f"€ {loan.amount}")
                    # Сплатити кредит
                    ui.button(
                        "Сплатити",
                        on_click=lambda this_loan=loan:
pay_loan(this_loan),
                    )

@ui.refreshable
def owned_trucks_view():
    if not State.owned_trucks:
        ui.label("Немає придбаних вантажівок.")
        return
    with ui.grid(columns=3).classes("w-full"):
        # Для кожної вантажівки у придбаних
        for truck in State.owned_trucks:
            # Створити картку
            with ui.card().tight():
                # Обрахувати шлях до файлу картинки
                image_path: str = os.path.join(
                    Const.CURRENT_FOLDER, "images", f"{truck.picture}.jpg"
                )
                # Вивести картинку
                ui.image(image_path)
            with ui.card_section():
                # Назва вантажівки
                ui.label(truck.name).classes("text-lg font-medium")
                # Категорія вантажівки
                ui.label(truck.category)
                with ui.row().classes("flex flex-row gap-2 mt-4"):
                    # Обрати вантажівку
                    ui.button(
                        "Обрати",

```

```

                                on_click=lambda this_truck=truck:
select_truck(this_truck),
                                )
                                # Продати вантажівку
                                ui.button(
                                    "Продати",
                                on_click=lambda this_truck=truck:
sell_truck(this_truck),
                                )

@ui.refreshable
def store_view():
    with ui.grid(columns=3).classes("w-full"):
        # Для кожної вантажівки у вантажівках
        for truck in State.trucks:
            # Створити картку
            with ui.card().tight():
                # Обрахувати шлях до файлу картинки
                image_path: str = os.path.join(
                    Const.CURRENT_FOLDER, "images", f"{truck.picture}.jpg"
                )
                # Вивести картинку
                ui.image(image_path)
            with ui.card_section().classes("w-full"):
                # Назва вантажівки
                ui.label(truck.name).classes("text-lg font-medium")
                # Категорія вантажівки
                ui.label(truck.category)
            with ui.row().classes("justify-between flex flex-row
w-full mt-4"):
                # Вартість вантажівки
                ui.label(f"€ {truck.price}").classes("font-bold ")
                # Купити вантажівку
                ui.button(
                    "Купити",
                                on_click=lambda this_truck=truck:
buy_truck(this_truck),
                                )

@ui.refreshable
def bank_view():
    with ui.grid(columns=2).classes("w-full"):
        # Для кожного кредиту зі всіх кредитів
        for loan in State.loans:
            # Створити картку
            with ui.card().tight():
                with ui.card_section():
                    # Назва кредиту
                    ui.label(f"Кредит у {loan.bank}").classes("font-medium
text-lg")

                    # Тривалість кредиту
                    ui.label(convert_months_to_years(loan.duration))
                with ui.row().classes("mt-4"):
                    # Вартість кредиту
                    ui.label(f"€ {loan.amount}")
                    # Взяти кредит
                    ui.button(
                        "Взяти",

```

```

                                on_click=lambda this_loan=loan:
take_loan(this_loan),
                                )

@ui.refreshable
def orders_view():
    # Зробити віджет мапи
    try:
        map = ui.leaflet(center=State.orders[0].coordinates, zoom=10)
    except IndexError:
        map = ui.leaflet(center=(47.839790847405894, 35.13965215348557),
zoom=10)

    with ui.grid(columns=3).classes("w-full"):
        # Для кожного замовлення у всіх замовленнях
        for order in State.orders:
            # Зробити картку
            with ui.card():
                with ui.card_section().classes("w-full"):
                    # Пункт призначення замовлення
                    ui.label(order.location).classes("text-lg font-
medium")

                    # Вартість замовлення
                    ui.label(f"€ {order.price}").classes("mb-4")
                    # Виконати замовлення
                    ui.button(
                        "Виконати",
                        on_click=lambda this_order=order: perform_order(
                            this_order, map
                        ),
                    )

State.trucks = load_trucks()
State.loans = load_loans()
State.orders = load_orders()

with ui.row().classes("flex flex-row gap-4 self-end"):
    selected_truck_label = ui.label(
        f"🚚 {State.selected_truck.name}" if State.selected_truck else ""
    )
    money_label = ui.label(f"💰 {State.money}")
with ui.tabs().classes("w-full") as main_tabs:
    loans_tab = ui.tab(TabName.LOANS, icon="o_credit_score")
    owned_tab = ui.tab(TabName.OWNED, icon="o_warehouse")
    store_tab = ui.tab(TabName.STORE, icon="o_store")
    bank_tab = ui.tab(TabName.BANK, icon="o_account_balance")
    orders_tab = ui.tab(TabName.ORDERERS, icon="o_list_alt")
with ui.tab_panels(tabs=main_tabs, value=store_tab).classes("w-full"):
    with ui.tab_panel(loans_tab):
        loans_view()
    with ui.tab_panel(owned_tab):
        owned_trucks_view()
    with ui.tab_panel(store_tab):
        store_view()
    with ui.tab_panel(bank_tab):
        bank_view()
    with ui.tab_panel(orders_tab):
        orders_view()

```

```
if __name__ in {"__main__", "__mp_main__"}:
    ui.run(title="Магазин вантажівок", favicon="🚚")
```

Текст додаткового модулю data.py

```
from dataclasses import dataclass
from peewee import SqliteDatabase, Model, CharField, IntegerField

class TabName:
    LOANS = "Кредити"
    OWNED = "Гараж"
    STORE = "Автосалон"
    BANK = "Банк"
    ORDERS = "Замовлення"

class TruckCategory:
    LOCAL = "Бус"
    LORRY = "Вантажівка"
    OFFROAD = "Всюдихід"

class BankName:
    PRIVAT = "Приват Банк"
    RAIFFEISEN = "Райфайзен Банк"
    CREDIT_AGRICOLE = "Кредит Агріколь"
    UNIVERSAL = "Банк Універсал"
    UKR_GAS_BANK = "Укргазбанк"

@dataclass
class ITruck:
    name: str
    category: TruckCategory
    price: int
    picture: str

@dataclass
class ILoan:
    amount: int
    duration: int
    bank: BankName

@dataclass
class IOrder:
    price: int
    location: str
    coordinates: tuple[float, float]

db_name = "database.db"
db = SqliteDatabase(db_name)

class BaseModel(Model):
    class Meta:
        database = db
```

```

class Truck(BaseModel):
    name = CharField()
    category = CharField()
    price = IntegerField()
    picture = CharField()

class Loan(BaseModel):
    amount = IntegerField()
    duration = IntegerField()
    bank = CharField()

class Order(BaseModel):
    price = IntegerField()
    location = CharField()
    coordinates = CharField()

db.connect()

def load_trucks() -> list[ITruck]:
    trucks: list[ITruck] = list()

    try:
        all_trucks_query = Truck.select()
    except Exception:
        print("ERROR: Failed to fetch the database.")
        return
    for row in all_trucks_query:
        truck = ITruck(
            name=row.name, category=row.category, price=row.price,
picture=row.picture
        )
        trucks.append(truck)

    return trucks

def load_loans() -> list[ILoan]:
    loans: list[ILoan] = list()

    try:
        all_loans_query = Loan.select()
    except Exception:
        print("ERROR: Failed to fetch the database.")
        return
    for row in all_loans_query:
        loan = ILoan(amount=row.amount, duration=row.duration, bank=row.bank)
        loans.append(loan)

    return loans

def load_orders() -> list[IOOrder]:
    orders: list[IOOrder] = list()

    try:
        all_orders_query = Order.select()
    except Exception:
        print("ERROR: Failed to fetch the database.")

```

```

        return
    for row in all_orders_query:
        order = IOrder(
            price=row.price,
            location=row.location,
            coordinates=tuple(
                float(coordinate) for coordinate in
row.coordinates.split(",")
            ),
        )
        orders.append(order)

    return orders

```

ДОДАТОК В
Слайди презентації

Продаж Вантажівок

Дипломна робота

доцент, Степаненко О.О.

студент КНТ-122, Онищенко О.А.

Рисунок 1 — Слайд 1

Зміст

01

Презентація складається з таких частин:

1. Порівняння фреймворків
2. Порівняння середовищ розробки
3. Огляд прикладів
4. Приклади використання

Рисунок 2 — Слайд 2

Порівняння фреймворків

02

Критерій	NiceGUI	Streamlit	Tkinter
Платформа	Веб-браузер	Веб-браузер	Десктоп
Персоналізація	Висока	Низька	Середня
Швидкість роботи	Висока	Середня	Середня

Рисунок 3 — Слайд 3

Порівняння середовищ розробки

03

Критерій	Visual Studio Code	Notepad	neovim
Зрозумілість інтерфейсу	Середня	Висока	Низька
Легкість освоєння	Середня	Висока	Низька
Зручність	Висока	Середня	Середня
Додаткові плагіни	Висока	Низька	Середня
Підтримка спільноти	Висока	Низька	Середня

Рисунок 4 — Слайд 4

Застосунок приклад: AutoRIA

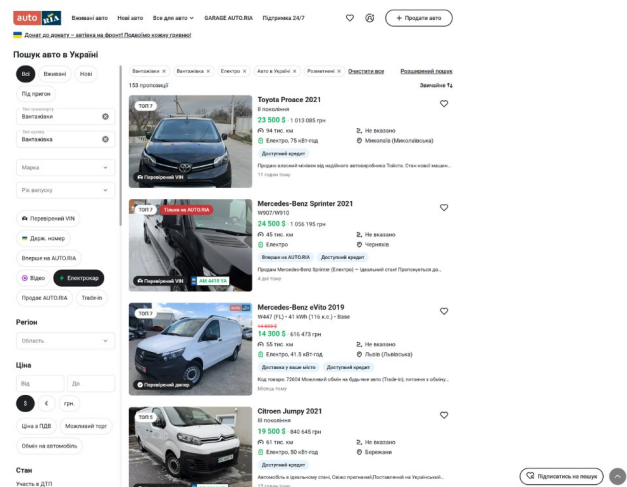


Рисунок 5 — Слайд 5

Застосунок приклад: Volvo

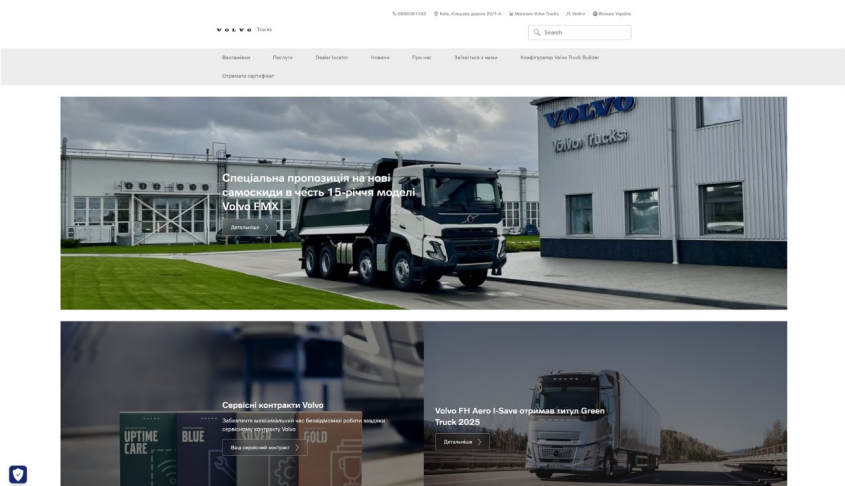


Рисунок 6 — Слайд 6

Застосунок приклад: DAF

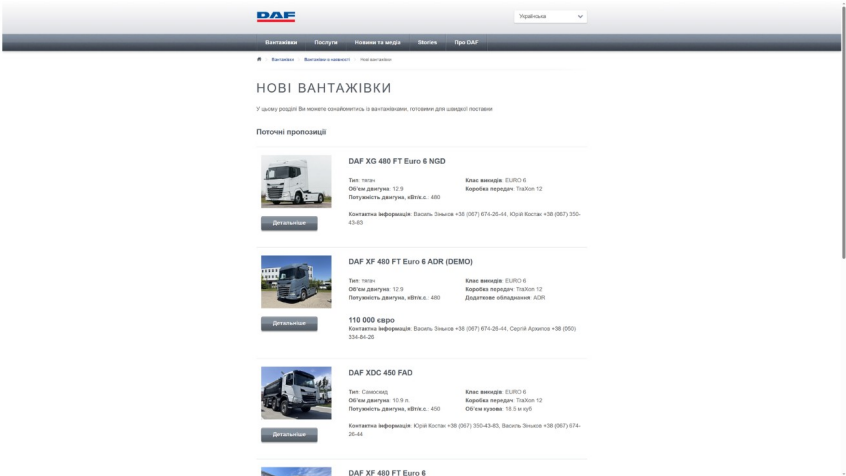


Рисунок 7 — Слайд 7

Застосунок приклад: RST

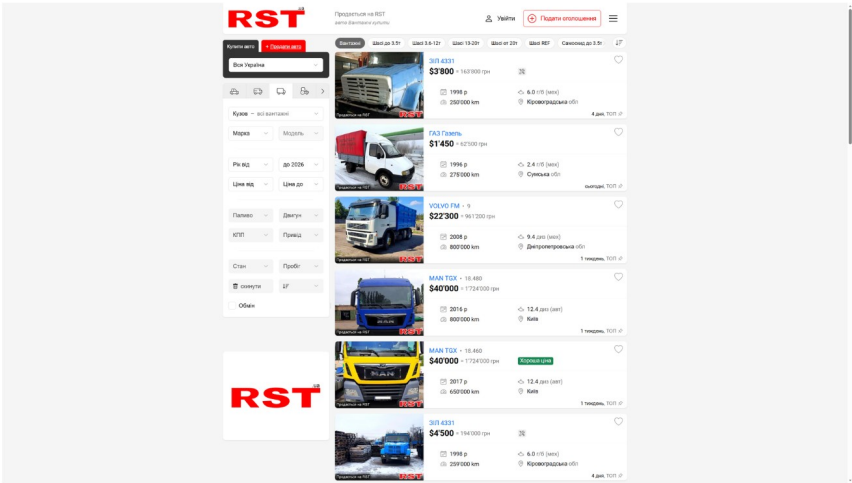


Рисунок 8 — Слайд 8

Застосунок приклад: OLX

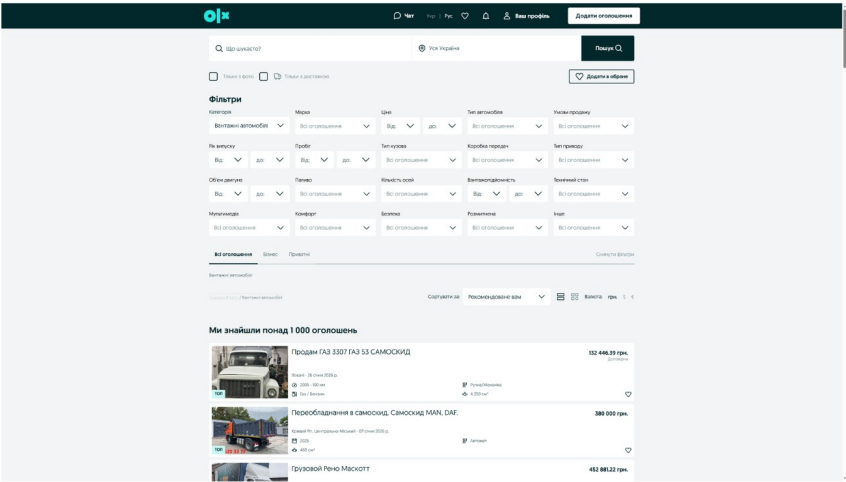


Рисунок 9 — Слайд 9

Вигляд застосунку: кредити

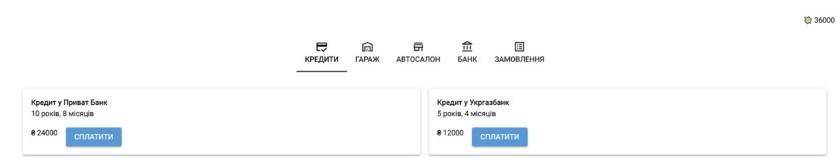


Рисунок 10 — Слайд 10

Вигляд застосунку: гараж

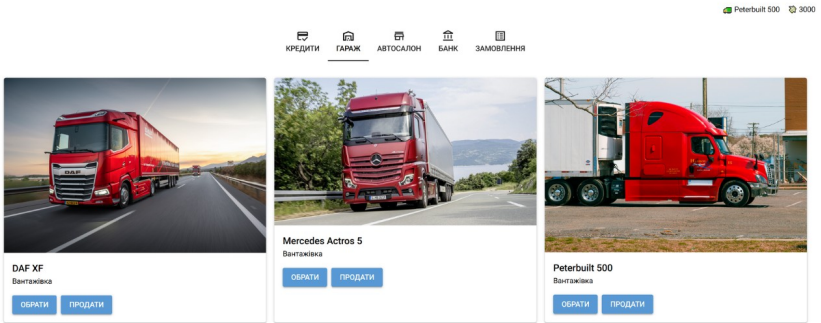


Рисунок 11 — Слайд 11

Вигляд застосунку: автосалон

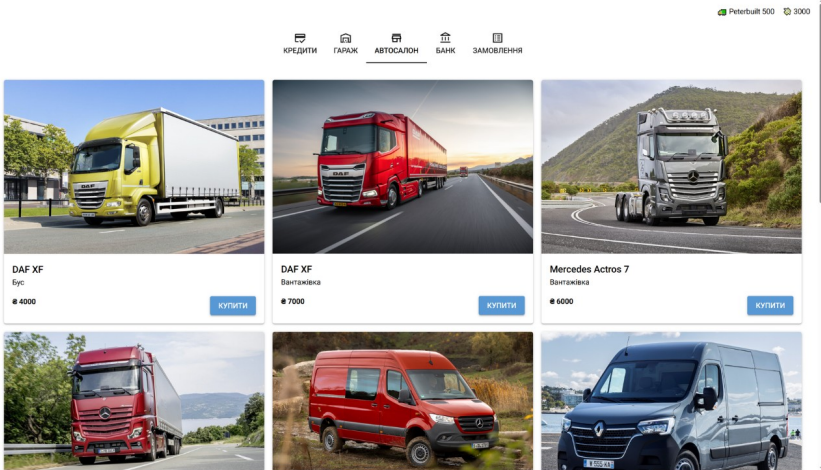


Рисунок 12 — Слайд 12

Вигляд застосунку: банк

12

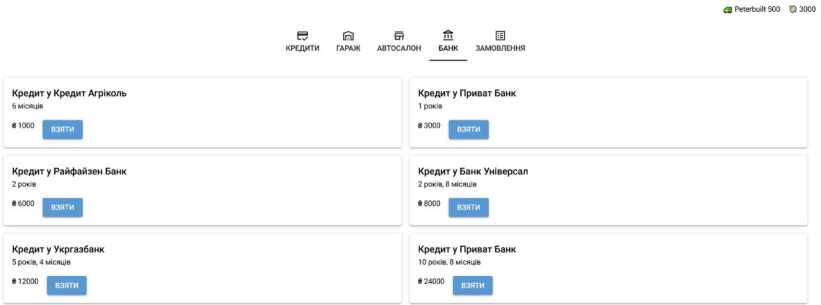


Рисунок 13 — Слайд 13

Вигляд застосунку: замовлення

13

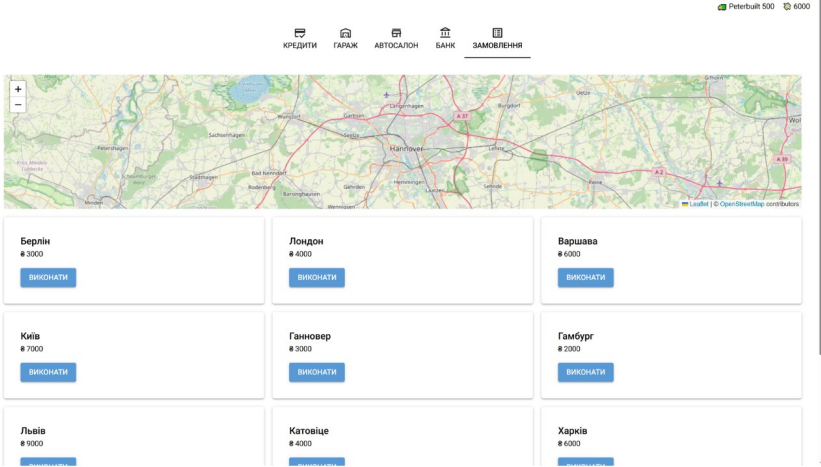


Рисунок 14 — Слайд 14

Висновки

14

**Так бо Бог полюбив світ, що дав
Сина Свого Однородженого, щоб
кожен, хто вірує в Нього, не
згинув, але мав життя вічне.**

Івана 3:16

Рисунок 15 — Слайд 15